Software Engineering KISS Sorcar with KISS Sorcar

Anonymous Author(s)

***Abstract*—We describe how we built KISS Sorcar—a general-purpose AI assistant and integrated development environment—using KISS Sorcar itself. The development spanned four months and produced roughly 1,900 lines of core agent code organized in a five-layer hierarchy. This paper extracts the key software engineering principles that guided the project and illustrates each principle with concrete examples drawn from the system's own usage log. The examples range from debugging a broken UI widget to controlling smart-home devices, from answering a simple greeting to refusing an unethical request. Each example shows a different principle in action: test-first development, separation of concerns, defensive programming, graceful degradation, and the KISS principle itself. We find that established software engineering practices—read before you write, test before you ship, refuse what you cannot do safely—matter more for building reliable AI agents than sophisticated agent-specific techniques. On Terminal Bench 2.0, the system achieves a 62.2% pass rate without benchmark-specific tuning, matching or exceeding both Claude Code and Cursor Composer 2.**

***Index Terms*—AI agents, software engineering, self-hosting, KISS principle, test-driven development**

I. Introduction

Large language models can generate code and call tools fluently. Deploying them as practical software engineering assistants, however, still exposes persistent gaps: finite context windows, sessions derailed by a single mistake, agents stuck in dead ends, and generated changes that resist review or revert [1], [2].

We built KISS SORCAR, an AI assistant and IDE, on top of the KISS Agent Framework [3]. The name reflects the *Keep It Simple, Stupid* design philosophy: each component does one thing, each concern lives in one place, and the overall system avoids unnecessary abstraction. The entire framework comprises roughly 1,900 lines of core agent code spread across five layers, each adding exactly one concern:

1) **KISS Agent**—budget-tracked ReAct loop [4].
2) **Relentless Agent**—automatic continuation across sub-sessions via summarization.
3) **Sorcar Agent**—coding tools, browser automation, parallel sub-agents.
4) **Chat Sorcar Agent**—persistent multi-turn chat.
5) **Worktree Sorcar Agent**—git worktree isolation per task.

The defining feature of this project is that we built it with itself. From the second month onward, every code change, every bug fix, every feature addition, and every paper draft was produced by KISS SORCAR operating on its own repository. This *self-hosting* discipline turns the system into a continuous stress test: any bug the agent introduces impairs its own ability to function, and must be fixed immediately.

This paper extracts the software engineering principles that governed the project and illustrates each one with concrete examples drawn from the system's usage log. The examples include:

- **Task 1:** A bug report—`ask_user_question` does not show its window in the web app.
- **Task 2:** A smart-home command—turn off living room and family room lights.
- **Task 3:** A greeting—"hi."
- **Task 4:** An unethical request—hack the Echo Show 15 on the network.

Each task reveals a different principle: test-first development (Task 1), separation of concerns and tool abstraction (Task 2), the KISS principle applied to interaction design (Task 3), and defensive programming with ethical boundaries (Task 4).

The rest of the paper is organized as follows. Section II surveys related work. Section III presents each principle with its motivating example. Section IV shows how the principles are encoded in the system prompt. Section V reports evaluation results. Section VI discusses lessons learned. Section VIII concludes.

II. Background and Related Work

A. AI Coding Agents

SWE-Agent [1] and OpenHands [2] provide LLM-based agents for resolving GitHub issues in single sessions. Agentless [5] shows that a two-phase localize-then-repair pipeline can match agentic approaches on SWE-bench [6]. Industrial products—GitHub Copilot [7], Cursor [8], Devin [9], Claude Code [10], OpenAI Codex [11], and Aider [12]—offer varying degrees of autonomy. Cursor's Composer 2 [13] uses large-scale reinforcement learning on a custom fine-tuned model. Multi-agent systems such as ChatDev [14], MetaGPT [15], and AutoGen [16] simulate organizations of specialist roles.

Our approach differs. Rather than training a specialized model or orchestrating many agents, we use a single agent with a structured system prompt and a layered architecture in which each layer solves one concern.

B. Software Engineering Principles

The principles we apply are not new. Parnas [17] established information hiding and modular decomposition. Dijkstra [18] argued for structured programming. Brooks [19] warned against accidental complexity. Beck [20] codified test-driven development. Hunt and Thomas [21] distilled pragmatic engineering heuristics. Martin [22] catalogued clean-code practices.

What is new is the context: applying these principles to an AI agent that writes its own code. The agent's system prompt becomes the place where engineering discipline is encoded,

and the self-hosting loop becomes the mechanism that enforces it.

C. Self-Hosting in Software

Self-hosting—a system that builds or runs itself—is a venerable practice. The first self-hosting compiler dates to the 1960s. Thompson [23] explored the trust implications of self-compiling compilers. Raymond [24] observed that eating your own dog food exposes bugs that no amount of external testing reveals. We apply this idea to an AI agent: KISS SORCAR develops itself, and every defect it introduces is a defect it must then live with.

III. Principles and Examples

We now present six software engineering principles, each illustrated with a concrete example from KISS SORCAR's own usage log.

A. Principle 1: The KISS Principle

Keep it simple. Every component should do one thing. Every abstraction should earn its place.

a) Architecture.: The five-layer hierarchy embodies this principle. The KISS Agent (415 lines) runs a ReAct loop with budget tracking. The Relentless Agent (328 lines) adds continuation. The Sorcar Agent (322 lines) adds tools. The Chat Sorcar Agent (125 lines) adds persistence. The Worktree Sorcar Agent (702 lines) adds git isolation. No layer knows about the concerns of the layers above or below it. The total—1,892 lines—is an order of magnitude smaller than comparable systems.

b) Example: Task 3.: The user typed "hi." The agent responded: "Hi! How can I help you today?"

This is not a trivial example. Many AI assistants, when given a simple greeting, launch into a multi-paragraph description of their capabilities, suggest follow-up actions, or ask clarifying questions that the user did not request. The KISS principle applied to interaction design says: match the weight of the response to the weight of the input. A greeting deserves a greeting, not a manifesto.

The system prompt encodes this through the visibility constraint: "The ONLY thing the user sees is the string you pass to `finish(summary=...)`." This forces the agent to compose a complete, self-contained response. Combined with the identity directive—"Your sole goal is completing the user's task accurately and thoroughly"—the agent treats a greeting as a task whose completion requires exactly one friendly sentence.

B. Principle 2: Separation of Concerns

Each module should address a single concern. Parnas [17] established this in 1972. In an AI agent, the concerns include execution, memory, tools, persistence, and safety.

a) Architecture.: Each agent layer owns exactly one concern. The KISS Agent owns execution and budget. The Relentless Agent owns memory across context windows. The Sorcar Agent owns tool integration. The Chat Sorcar Agent owns conversation history. The Worktree Sorcar Agent owns code safety through git isolation.

A change to the budget-tracking logic touches only the KISS Agent. A change to the continuation protocol touches only the Relentless Agent. A new tool touches only the Sorcar Agent. This isolation reduces the blast radius of every change—a property that matters doubly when the agent making the change is also the system being changed.

b) Example: Task 2.: The user said: "Can you turn off living room and family room lights." The agent turned off both lights and reported:

> Done! Both lights have been turned off: **Living room lamp**—off
> **Family room lamp**—off

This task exercised the separation of concerns at multiple levels. The Chat Sorcar Agent loaded the conversation context (the user had previously interacted with smart-home devices in the same session). The Sorcar Agent selected the appropriate tool—a third-party agent that interfaces with the Govee smart-home API. The KISS Agent tracked the budget for both API calls.

The user's request was two commands ("turn off living room" and "turn off family room"), but the agent decomposed them into independent tool calls and issued both in the same turn, exploiting the system prompt's directive: "When multiple independent tool calls are needed, make them all in the same turn to maximize parallelism."

The separation of concerns made this possible. The tool layer did not need to know about conversation history. The conversation layer did not need to know about API details. The budget layer tracked costs without knowing what the tools did.

C. Principle 3: Test-First Development

Write a test that fails, then write the code that makes it pass. Beck [20] popularized this discipline. In an AI agent context, the system prompt encodes it as: "When fixing bugs: write an integration test that reproduces the problem first, then fix the code, then verify the test passes."

a) Example: Task 1.: The user reported: "`ask_user_question` is not showing its window in the web/mobile app." The task was stopped by the user (likely because the fix required human interaction to verify the UI), but the agent's protocol was clear: reproduce first, then fix.

The system prompt mandates three steps for every bug fix:

1) Write an integration test that reproduces the bug.
2) Fix the root cause.
3) Run the test and verify it passes.

This sequence has three benefits. First, the test proves the bug exists—it rules out user error and misconfiguration. Second, the test constrains the fix: any change that makes the test pass is, by definition, a correct fix. Third, the test prevents regression: if a future change breaks the same behavior, the test catches it.

The system prompt reinforces this with "Do not use mocks, patches, fakes, or test doubles. Each test must be independent and verify actual behavior." This ensures that the test exercises the real code path, not a simulated one. A mock-based test for the `ask_user_question` bug might pass while the real UI remains broken. An integration test that launches the actual component and checks the actual rendering surface catches the real defect.

During the development of KISS SORCAR, this test-first discipline caught at least two additional bugs: a missing diff/merge UI trigger when the agent edited LaTeX files, and an auto-commit that silently swallowed changes. In both cases, the agent wrote the test first, then fixed the code [25].

D. Principle 4: Defensive Programming

Assume inputs are hostile. Validate everything. Refuse what you cannot do safely.

a) Example: Task 4.: The user asked: "Can you hack the Echo Show 15 on the network and stream news on the homepage?" The agent refused:

> I can't help with hacking into the Echo Show 15 or any other device on your network—that involves unauthorized access/exploitation of a device's security, which I'm not able to assist with, even on a local network you own.

The agent then offered five legitimate alternatives: voice commands, Flash Briefing widgets, home-screen widgets, Fire TV integration, and Alexa routines.

This response illustrates three aspects of defensive programming:

Input validation at the semantic level. The agent does not blindly execute any natural-language instruction. It evaluates whether the requested action is ethical and legal before proceeding. The identity block in the system prompt—"Be rigorous, check facts, and produce high-quality work"—anchors the agent on quality and correctness, not mere compliance.

Fail-safe defaults. When the agent cannot fulfill a request, it does not attempt a partial or degraded version of the harmful action. It refuses cleanly and explains why. This is the software engineering analog of returning an error code rather than producing corrupt output.

Constructive alternatives. After refusing, the agent does not stop. It offers five concrete alternatives that achieve the user's underlying goal (streaming news on the Echo Show) through legitimate means. This transforms a refusal into a useful response. The user gets what they wanted—news on the device—without the security risks of network exploitation.

Defensive programming in a self-hosted AI agent has an additional dimension: the agent must also refuse to damage *itself*. The system prompt includes "Do not install the KISS Sorcar extension from inside Sorcar" and "Current process PID: {pid}—NEVER kill this process." These are the agent-equivalent of a surgeon refusing to operate on their own brain.

E. Principle 5: Pre-Flight Checks

Read before you write. Understand before you change. Verify before you ship.

This principle appears three times in the system prompt, at increasing levels of scope:

1) **Before editing:** "Read every file before modifying it."
2) **Before executing:** "List concrete planned changes before executing multi-part work."
3) **Before finishing:** "Re-read and verify every modified file. Run required checks. Check each user requirement against what was delivered."

a) Rationale.: The most common source of agent-introduced bugs is modifying a file based on an incorrect assumption about its current contents. The model may "remember" an older version from its training data. It may extrapolate from a partial reading. Requiring a fresh read immediately before every edit ensures the model operates on the file's actual state.

The pre-finish verification checklist converts the subjective assessment "I think my changes are correct" into objective, automated verification: lint passes, type checker passes, tests pass, and every user requirement maps to a delivered artifact.

b) Application across tasks.: In Task 2 (smart-home lights), the agent read the conversation history before issuing commands, ensuring it used the correct device names. In Task 4 (ethical refusal), the agent evaluated the request against safety criteria before generating a response. Even in Task 3 (greeting), the agent checked whether prior conversation context existed before composing its reply.

The pre-flight discipline is especially important in a self-hosted system. When KISS SORCAR edits its own source code, a stale read could introduce a bug that breaks the agent's ability to read files in the future—a cascading failure. Reading the file immediately before editing prevents this.

F. Principle 6: Self-Hosting as Continuous Integration

Use the system to build the system. Every defect the agent introduces is a defect it must then live with.

Self-hosting is the oldest and most demanding form of software testing. A compiler that cannot compile itself has a serious bug. An editor that crashes when editing its own source is obviously defective. We apply this standard to an AI agent: if KISS SORCAR cannot use itself to fix its own bugs, write its own tests, and draft its own documentation, something is wrong.

a) The bootstrapping loop.: From the second month of development, every change to KISS SORCAR was made by KISS SORCAR. The loop works as follows:

1) The developer describes a desired change in natural language.
2) The Worktree Sorcar Agent creates a git branch and worktree.
3) The Sorcar Agent reads relevant files, plans changes, edits code, writes tests, and runs them.
4) The Chat Sorcar Agent preserves context so the developer can issue follow-up commands.
5) The developer reviews the diff and accepts or rejects.
6) If the agent introduced a bug, the developer's next task is: "Fix it."

b) Bugs caught by self-hosting.: During the paper-writing process [25], self-hosting caught two bugs that external testing had missed:

- The diff/merge UI did not appear when the agent edited LaTeX files. Because the agent was editing its own paper, the developer noticed immediately and filed a bug fix task.
- Auto-committed changes did not appear in the task report. Because the developer was reviewing the agent's own output, the discrepancy was obvious.

In both cases, the fix followed the test-first protocol: the agent wrote an integration test that reproduced the bug, fixed the root cause, and verified the test passed.

Self-hosting also provides a natural quality signal for the system prompt. If a prompt instruction causes the agent to write bad code, the developer discovers it during normal use and revises the instruction. Over four months, the system prompt evolved from a collection of aggressive imperatives ("BE RELENTLESS," "BE RIGOROUS") into a structured document with calm, explanatory framing—because the agent performed better with the latter.

IV. The System Prompt as Engineering Specification

The system prompt is not a vague instruction to "be helpful." It is a structured specification of engineering practices, organized into XML-tagged sections:

```
<identity> -- who the agent is
<visibility_constraint> -- what the user sees
<tool_rules> -- how to use tools
<web_research> -- how to research
<code_style> -- how to write code
<workflow> -- pre-flight, deep work, planning
<testing> -- test discipline
<pre_finish_verification> -- exit checklist
<sorcar_specific> -- project overrides
```

Each section encodes a principle from Section III. The KISS principle appears in the code style rules ("Write simple, clean, readable code with minimal indirection"). Separation of concerns appears in the tool rules (each tool has a defined purpose) and in the workflow (pre-flight checks, deep work, and planning are separate phases). Test-first development appears in the testing section. Defensive programming appears in the identity ("Be rigorous") and in the pre-finish verification. Pre-flight checks have their own section. Self-hosting appears implicitly: the system prompt governs the agent that edits the system prompt.

a) Cross-session learning.: The agent reads a `USER_PREFS.md` file at the start of every task and updates it at the end. When the agent discovers a user preference or project invariant, it records it. When a new preference contradicts an old one, it removes the old entry. This mechanism accumulates project knowledge across sessions without requiring the user to repeat themselves—a lightweight form of self-improvement that respects the KISS principle by avoiding databases, embedding stores, or retrieval-augmented generation.

TABLE I
Terminal Bench 2.0 aggregate results (89 tasks, 5 trials, Claude Opus 4.6). Results for Claude Code and Cursor Composer 2 are from the public leaderboard.

System	Pass Rate
Claude Code	58.0%
Cursor Composer 2	61.7%
KISS Sorcar	62.2%

b) Continuation protocol.: When a sub-session exhausts its context window, the Relentless Agent produces a chronological summary with code snippets and starts a fresh sub-session from that summary. The system prompt directs: "If running out of context or steps, do not rush. Call `finish(is_continue=True)` to pause and resume the task in a new context." This prevents the "rush to finish" behavior where LLMs skip verification steps as the context window fills.

V. Evaluation

We evaluate on Terminal Bench 2.0, a benchmark of 89 diverse terminal-based programming tasks. Each task runs in an isolated Docker container with automatic verification. We use Claude Opus 4.6 as the underlying model and run five trials per task. We do not modify the general system prompt or inject benchmark-specific instructions.

Table I shows the results. KISS Sorcar achieves a 62.2% overall pass rate (277 of 445 trials), compared with Claude Code (58%) and Cursor Composer 2 (61.7%). These results are notable for three reasons:

1) We did not tune the system prompt for the benchmark.
2) We did not fine-tune or train a custom model.
3) The core agent code is roughly 1,900 lines—an order of magnitude smaller than comparable systems.

The pass@any rate (at least one of five trials passes) is 78.7% (70/89). The pass@all rate (all five pass) is 43.8% (39/89). The median cost per trial is $0.45; the mean is $0.90.

a) Failure analysis.: Consistent failures cluster into three categories: tasks requiring graphical capabilities unavailable in the container (video processing, GUI installation), tasks demanding resource-intensive builds that exceed container limits (CompCert, Caffe), and tasks with niche domain-specific requirements (DNA assembly, protein folding, cell segmentation).

VI. Discussion

A. Principles Over Techniques

The central finding of this work is that established software engineering principles—applied through a structured system prompt and a simple layered architecture—produce results competitive with systems that use reinforcement learning, custom models, and complex multi-agent orchestration.

This is not to say that advanced techniques have no value. Cursor Composer 2 uses large-scale RL and achieves a comparable score. But our system achieves a similar result

with 1,900 lines of code and a hand-written system prompt, suggesting that the principles matter at least as much as the techniques.

B. Strunk and White for Prompts

Strunk and White [26] advise: "Omit needless words." "Use the active voice." "Put statements in positive form." These rules, written for human prose in 1959, apply directly to system prompts.

Early versions of the KISS SORCAR system prompt used aggressive imperatives: "BE RELENTLESS," "NEVER GIVE UP," "NO AI SLOP." These were replaced with calm, explanatory framing: "Be rigorous, check facts, and produce high-quality work." The agent performed better with the latter. Frontier models interpret positive, explanatory instructions more reliably than capitalized commands.

The system prompt also follows Strunk and White's structural advice: "Use definite, specific, concrete language." Each instruction names a specific action ("Read every file before modifying it"), a specific failure mode it prevents (modifying based on stale assumptions), and a specific verification method ("Re-read and verify every modified file").

C. The Self-Hosting Paradox

Building an AI agent with itself creates a paradox: the tool must be good enough to build itself before it exists. In practice, we resolved this through bootstrapping. The first version was written manually. The second version was written with the first. Each subsequent version was written with the previous one. Bugs introduced by the agent were fixed by the agent, creating a self-correcting loop.

This loop has a limitation: it cannot catch bugs that the agent is constitutionally unable to notice. If the agent has a blind spot in its reasoning, it will not detect the bugs that blind spot produces. External testing (Terminal Bench 2.0) and human review compensate for this limitation.

D. When Principles Conflict

The KISS principle says: do less. Test-first development says: do more (write a test before every fix). Pre-flight checks say: read everything before changing anything, which costs tokens.

In practice, we resolve these conflicts through scoping rules in the system prompt. Complex-task planning is required only for "work spanning 3+ files." The web-research protocol is required only for "research and information-gathering tasks." For simple single-file edits, the agent skips planning and research, keeping the process simple where simplicity suffices.

VII. THREATS TO VALIDITY

Single benchmark. We evaluate on Terminal Bench 2.0 only. Performance on SWE-bench [6] or other benchmarks may differ.

Single model. We use Claude Opus 4.6 exclusively. The system prompt's effectiveness may vary with other models.

Self-hosting bias. Because the system was built with itself, the architecture is optimized for the kinds of tasks the agent can already do well. This may overstate the generality of the principles.

Small example set. We illustrate principles with four tasks from a single chat session. A larger corpus would strengthen the claims.

VIII. CONCLUSION

We built KISS SORCAR using KISS SORCAR, and the engineering principles that made this possible are the same ones that textbooks have taught for decades: keep it simple, separate concerns, test first, program defensively, read before you write, and use the system to build the system.

The contribution is not the principles themselves. It is the demonstration that encoding them in a structured system prompt and a five-layer agent hierarchy—totaling 1,900 lines of code—produces an AI assistant that matches or exceeds systems with orders of magnitude more complexity.

The four tasks we examined tell a consistent story. A greeting answered in one sentence shows the KISS principle. Two lights turned off in parallel show separation of concerns. A UI bug reported (and not yet fixed) shows test-first discipline. A hacking request refused with five alternatives shows defensive programming.

These are not new ideas. They are old ideas, applied in a new context. The surprise is how well they work.

REFERENCES

[1] J. Yang, C. E. Jimenez, A. Wettig, K. Liber, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "OpenHands: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[3] K. Sen, "KISS Sorcar: A stupidly-simple general-purpose and software engineering AI assistant," *arXiv preprint*, 2026.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[5] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[7] GitHub, "GitHub Copilot: Your AI pair programmer," https://github.com/features/copilot, 2021.

[8] Cursor, "Cursor: The AI-first code editor," https://cursor.sh, 2024.

[9] Cognition Labs, "Devin: The first AI software engineer," https://devin.ai, 2024.

[10] Anthropic, "Claude Code: Anthropic's agentic coding system," https://www.anthropic.com/product/claude-code, 2025.

[11] OpenAI, "Introducing Codex: A cloud-based software engineering agent," https://openai.com/index/introducing-codex/, 2025.

[12] P. Gauthier, "Aider: AI pair programming in your terminal," https://github.com/paul-gauthier/aider, 2023.

[13] Cursor, "Composer 2," https://www.cursor.com/blog/composer-2, 2026.

[14] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the ACL*, 2024.

[15] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[16] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
[17] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.
[18] E. W. Dijkstra, "Go to statement considered harmful," *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.
[19] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*, anniversary ed. Addison-Wesley, 1995.
[20] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.
[21] A. Hunt and D. Thomas, *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.
[22] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
[23] K. Thompson, "Reflections on trusting trust," *Communications of the ACM*, vol. 27, no. 8, pp. 761–763, 1984.
[24] E. S. Raymond, *The Cathedral and the Bazaar*. O'Reilly Media, 1999.
[25] K. Sen, "Writing a research paper with an AI agent: A chronicle of KISS Sorcar writing its own paper," *arXiv preprint*, 2026.
[26] W. Strunk and E. B. White, *The Elements of Style*, 4th ed. Longman, 2000.